

基于Isaac Gym仿真平台训练轮足机器人的控制算法

环境配置

硬件需求

使用Isaac Gym进行训练刚需一张支持CUDA功能的Nvidia显卡，为了流畅的进行训练以及可视化训练结果，推荐使用显存16G以上的RTX显卡。

系统配置

Isaac Gym的训练需要在Linux系统上进行，推荐使用Ubuntu20.04，并将Isaac Gym配置在conda虚拟环境中运行。

软件配置

安装Nvidia驱动

在Ubuntu20.04上安装Nvidia驱动，推荐安装550版本的驱动，使用 `nvidia-smi` 命令查看驱动是否安装成功。

```
gpz@ps:~$ nvidia-smi
Wed Dec 25 14:40:25 2024

+-----+
| NVIDIA-SMI 550.135                Driver Version: 550.135          CUDA Version: 12.4     |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf          Pwr:Usage/Cap|   /-          Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|  0   NVIDIA GeForce RTX 4080 ...    Off | 00000000:18:00.0 On  |      13%      Default  |
| 38%   34C   P8              19W / 320W | 502MiB / 16376MiB |      13%      Default  |
+-----+-----+-----+-----+-----+-----+
|  1   NVIDIA GeForce RTX 4080 ...    Off | 00000000:51:00.0 Off |       0%      Default  |
| 38%   32C   P8              12W / 320W | 11MiB / 16376MiB |       0%      Default  |
+-----+-----+-----+-----+-----+-----+
|  2   NVIDIA GeForce RTX 4080 ...    Off | 00000000:8A:00.0 Off |       0%      Default  |
| 38%   29C   P8              11W / 320W | 11MiB / 16376MiB |       0%      Default  |
+-----+-----+-----+-----+-----+-----+
|  3   NVIDIA GeForce RTX 4080 ...    Off | 00000000:C3:00.0 Off |       0%      Default  |
| 38%   32C   P8              14W / 320W | 14MiB / 16376MiB |       0%      Default  |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type   Process name                      GPU Memory |
|   ID   ID     ID              |    |              | Usage     |
+-----+-----+-----+-----+-----+-----+
|  0   N/A  N/A       29812     G   /usr/lib/xorg/Xorg                233MiB   |
|  0   N/A  N/A      30001     G   /usr/bin/gnome-shell              78MiB   |
|  0   N/A  N/A      42955     G   /usr/lib/firefox/firefox          25MiB   |
|  0   N/A  N/A      43387     G   ...ps/Downloads/v2rayN-linux-64/v2rayN 21MiB   |
|  0   N/A  N/A     213318     G   ...00,262144 --variations-seed-version 43MiB   |
|  0   N/A  N/A     283579     G   ...erProcess --variations-seed-version 46MiB   |
|  0   N/A  N/A     296425     G   ...an,WebOTP --variations-seed-version 18MiB   |
|  1   N/A  N/A       29812     G   /usr/lib/xorg/Xorg                 4MiB   |
|  2   N/A  N/A       29812     G   /usr/lib/xorg/Xorg                 4MiB   |
|  3   N/A  N/A       29812     G   /usr/lib/xorg/Xorg                 4MiB   |
+-----+-----+-----+-----+-----+-----+
```

安装CUDA

在Ubuntu20.04上安装CUDA11.4，安装完成后可以使用 `nvcc --version` 命令查看CUDA版本，如果显示CUDA版本信息，则安装成功。

```
gpz@ps:~$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Tue_Feb_27_16:19:38_PST_2024
Cuda compilation tools, release 12.4, V12.4.99
Build cuda 12.4.r12.4/compiler.33961263 0
```

安装miniconda

在Ubuntu20.04上安装miniconda，安装完成后可以使用 `conda --version` 命令查看miniconda版本，如果显示miniconda版本信息，则安装成功。

```
gpz@ps:~$ conda --version
conda 24.11.1
```

强化学习训练框架

1. 配置python虚拟环境（推荐使用Python3.8版本）。

- 创建虚拟环境：`conda create -n scut python=3.8`

```
gpz@ps:~$ conda create -n scut python=3.8
```

- 激活虚拟环境：`conda activate scut`

```
(scut) gpz@ps:~$ conda activate scut
(scut) gpz@ps:~$ python --version
Python 3.8.20
```

2. 安装pytorch 1.13.1和cuda-11.7

- 使用pip安装：`pip install torch==1.13.1+cu117 torchvision==0.14.1+cu117 --extra-index-url https://download.pytorch.org/whl/cu117`

```
(scut) gpz@ps:~$ pip install torch==1.13.1+cu117 \
> torchvision==0.14.1+cu117 \
> --extra-index-url https://download.pytorch.org/whl/cu117
```

3. 下载并安装Isaac Gym

- NVIDIA官网下载Isaac Gym，解压缩后得到 `isaacgym` 文件夹。
- 进入 `isaacgym/python` 文件夹，使用pip安装：`pip install -e .`

```
(scut) gpz@ps:~/Work/opensourceproj$ cd isaacgym/python/
(scut) gpz@ps:~/Work/opensourceproj/isaacgym/python$ pip install -e .
```

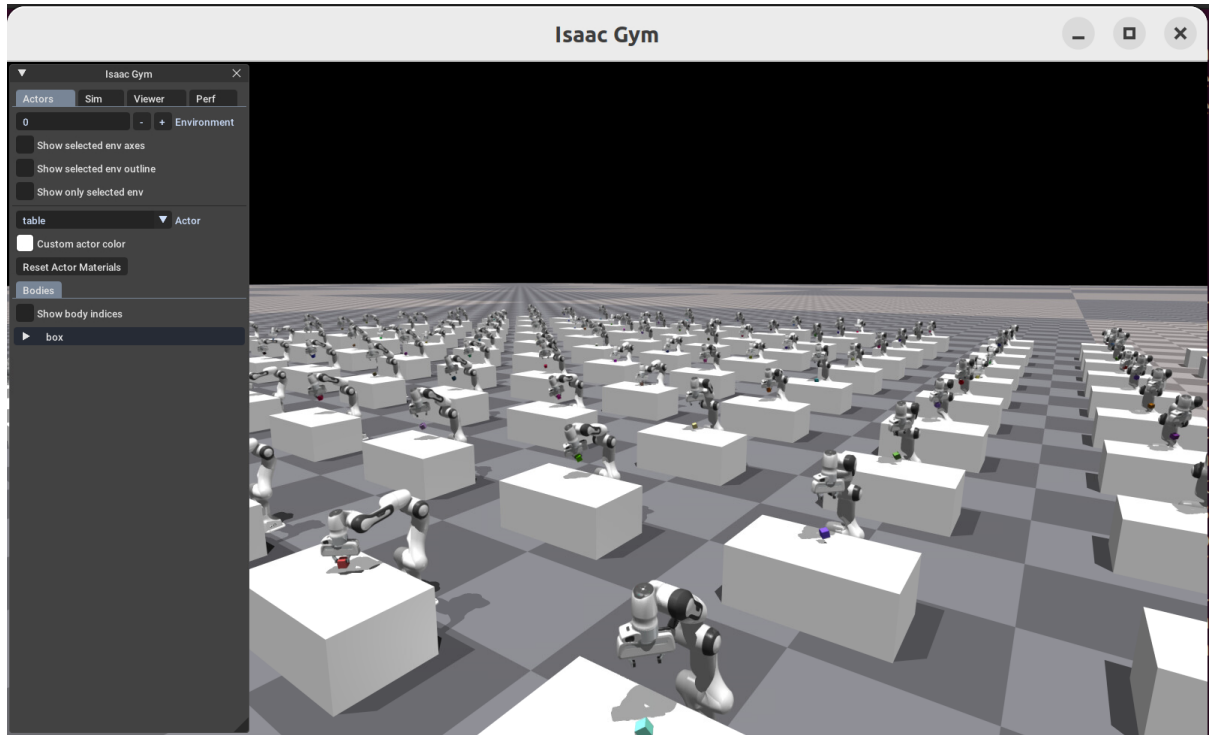
- 进入 `examples` 文件夹，运行示例代码进行验证：`python franka_cube_ik_osc.py`

```
(scut) gpz@ps:~/Work/opensourceproj/isaacgym/python$ cd examples/
(scut) gpz@ps:~/Work/opensourceproj/isaacgym/python/examples$ python franka_cube_ik_osc.py
```

- 如果出现 `ImportError: libpython3.8.so.1.0: cannot open shared object file: No such file or directory` 错误，则需要将python的路径添加到系统环境变量中。
- 使用命令：`export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/home/gpz/miniconda3/envs/scut/lib`

```
(scut) gpz@ps:~/Work/opensourceproj/isaacgym/python/examples$ python franka_cube_ik_osc.py
Importing module 'gym_38' (/home/gpz/Work/opensourceproj/isaacgym/python/isaacgym/_bindings/linux-x86_64/gym_38.so)
Traceback (most recent call last):
  File "franka_cube_ik_osc.py", line 16, in <module>
    from isaacgym import gymapi
  File "/home/gpz/Work/opensourceproj/isaacgym/python/isaacgym/_init_.py", line 5, in <module>
    from isaacgym import gymapi
  File "/home/gpz/Work/opensourceproj/isaacgym/python/isaacgym/gymapi.py", line 104, in <module>
    _import_active_version()
  File "/home/gpz/Work/opensourceproj/isaacgym/python/isaacgym/gymapi.py", line 63, in _import_active_version
    module = importlib.import_module(package_path)
  File "/home/gpz/miniconda3/envs/scut/lib/python3.8/importlib/_init_.py", line 127, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
ImportError: libpython3.8.so.1.0: cannot open shared object file: No such file or directory
(scut) gpz@ps:~/Work/opensourceproj/isaacgym/python/examples$ export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/home/gpz/miniconda3/envs/scut/lib
```

- 成功运行示例代码后，可以看到如下界面：



4. 安装训练工程

文件夹 `L2B_2wheel`，`L2B_4wheel`，`L2B_switch` 是三个训练工程，分别对应L2B两轮，四轮及模式切换。

- 一般在安装训练工程之前还需要安装 `rs1_rl`

“`rs1_rl`” 是实现强化学习算法（PPO）的库。

- 使用 Git 克隆仓库: `git clone https://github.com/leggedrobotics/sl_rl.git`。
- 切换到 v1.0.2 分支: `cd rs1_rl && git checkout v1.0.2`。
- 安装: `pip install -e .`。
- 华工的工程中附带了`rs1_rl`，所以省略了上一步，直接进入任意文件夹，使用命令 `pip install -e .` 安装训练工程，以 `L2B_switch` 为例：
 - 进入文件夹 `L2B_switch` 并使用`pip`安装，需要注意的是由于不同的工程是由不同的人负责的，使用不同工程时，需要在工程文件夹中再次使用 `pip install -e .` 安装：

```
(scut) gpz@ps:~/Work/SCUT$ cd L2B_switch/  
(scut) gpz@ps:~/Work/SCUT/L2B_switch$ pip install -e .
```

训练及Sim2Sim

参考[宇树开源机器人强化学习训练工程](#)

训练

以 `y1a_mode_switching` 任务为例。

运行命令进行训练：`python l2b_2wheel/scripts/train.py --task=y1a_mode_switching`。

训练时会自动保存过程模型到

`logs/y1a_mode_switching/<date_time>_<run_name>/model_<iteration>.pt`，同时会将当前的 `legged_robot_config.py`，`legged_robot.py` 和 `y1a_mode_switching_config.py`，`y1a_mode_switching.py` 四个文件保存到同一目录下。

Play

在 Gym 中查看训练效果，运行命令：`python l2b_2wheel/scripts/play.py --task=y1a_mode_switching --load_run=Nov29_09-07-54_`，查看 `Nov29_09-07-54_` 这个训练的效果。`--load_run` 后面的参数是训练时的文件夹名称，如果不指定，则默认加载最新的训练。

Play时会导出 Actor 网络 and Policy 网络，保存为

`logs/y1a_mode_switching/exported/policies/actor.pt` 和 `logs/y1a_mode_switching/exported/policies/policy_1.pt`。

这里有一个Bug，理论上Play不同文件夹的模型时，会自动加载文件夹中的四个 `.py` 文件，读取训练时的配置进行Play，但实际上需要手动将文件夹中的四个文件覆盖到原来的路径中，才能正确加载配置。

Sim2Sim (Mujoco)

在 Mujoco 仿真器中运行 Sim2Sim：`python l2b_2wheel/scripts/sim2sim.py --load_model=logs/y1a_mode_switching/exported/policies/policy_1.pt`

`sim2sim`是我们自己实现的，和宇树的不太一样。

实机部署 (Sim2Real)

`sim2sim`可以达到训练预期效果后即可进行实机部署。

一般来说，实机部署是将 `.pt` 模型文件转为 `.jit` 格式，借助ONNX进行模型部署。考虑到采用这种方法部署在arm版上需要依赖ONNX-ARM，而ONNX-ARM的编译过程比较复杂，因此华工采用了另一种方法：利用Matlab将 `.pt` 模型转化为不依赖任何第三方库的 C/C++ 代码，然后构建为 `libstand_mode_lib.so` 文件，在 RK3588上运行。

部署相关的代码在 `deploy` 文件夹下。

模型转换

将Play时生成的 `actor.pt` 拷贝到 `deploy/policies` 文件夹下，运行 `deploy/python` 文件夹中运行 `python transfer_weight.py`，生成 `actor.mat`。

```
(scut) gpz@ps:~/Work/SCUT$ cd deploy/python/
(scut) gpz@ps:~/Work/SCUT/deploy/python$ python transfer_weight.py
```

代码生成

进入 `deploy/matlab` 文件夹，使用 **Matlab R2023b** 依次运行 `ActorNet.m`，`assign_weight.m` 和 `generate_c.m`。需要安装 **MATLAB Coder Interface for Deep Learning** 组件

- 运行 `ActorNet.m`，训练一个与轮足机器人强化学习训练时相同网络大小的policy。
- 运行 `assign_weight.m` 脚本分配权重参数，将 `actor.mat` 中的参数覆盖policy中的参数，存为 `save_actor_policy.mat` 文件。
- 已经编写好了 `predictNet.m` 函数，该函数加载 `save_actor_policy.mat` 文件，用于深度学习网络的预测。
- 运行 `generate_c.m` 调用 Codegen 工具箱将 `predictNet.m` 函数生成为不依赖任何深度学习库的 C/C++ 代码。
- 生成的代码位于 `codegen\lib\predictNet` 文件夹，将所有的 `.h` 和 `.c` 文件（除 `main.c` 外）分别放到待编译项目的 `include` 和 `src` 文件夹下，配置好 `CMakeLists.txt` 文件，进行交叉编译即可生成 `.so` 文件

